

Inductive Graph-Based Multi-Agent Reinforcement Learning for Autonomous Cyber Defence

Silja Kluge^{1,2}, Jan Ernst¹, Thies Möhlenhof¹, Tobias Hürten¹,
Florian Spelter¹, Laurin Holz^{1,3}, Roberto Rigolin F. Lopes¹, and Johannes F. Loevenich^{1,4}

¹CortAIx Labs, Thales Deutschland, Ditzingen, Germany

²Eberhard Karls Universität Tübingen, Tübingen, Germany

³Department of Computer Science, University of Ulm, Ulm, Germany

⁴Department of Mathematics/Computer Science, University of Osnabrück, Osnabrück, Germany

Email: {silja.kluge, jan.ernst, thies.moehlenhof, tobias.huerten, roberto.rigolin, johannes.loevenich}@thaligroup.com

Abstract—This paper investigates an inductive, graph-based MARL approach for autonomous cyber defense in an enterprise network scenario. We model the defense problem as a decentralized partially observable Markov Decision Process (Dec-POMDP) and employ graph neural networks with Proximal Policy Optimization (PPO) to learn distributed defensive policies under limited observability and communication constraints. In addition, we introduce an expert-informed heuristic defense agent that serves as an interpretable reference policy. We apply Explainable Artificial Intelligence (XAI) techniques based on reward decomposition and action distribution analysis. Our results show that the proposed MARL approach achieves performance comparable to state-of-the-art benchmark solutions, while XAI methods reveal structural reward misalignment that biases learning toward reactive connectivity control over proactive situational awareness.

Index Terms—Autonomous Cyber Defence, Multi-Agent Reinforcement Learning, Graph Neural Networks, Explainable Artificial Intelligence

I. INTRODUCTION

The cyber threat landscape is evolving rapidly, driven by the combination of human expertise and recent advances in Multi Agent Reinforcement Learning (MARL) [1]–[4]. Artificial Intelligence (AI) has shown the capability to automate cyber attacks with near human-level effectiveness [5], significantly lowering the barrier for sophisticated adversarial operations [2], [3], [6]. At the same time, modern military and critical infrastructure networks are increasingly complex [7], highly segmented, and subject to dynamic operational priorities, rendering static or manually operated defenses insufficient.

To counter these challenges, Autonomous Cyber Defense (ACD) mechanisms are gaining importance, with MARL emerging as a promising paradigm for distributed and adaptive defense [8]–[10]. By decomposing the defense task across multiple agents operating under partial observability, MARL systems can scale to large networks with heterogeneous security zones while reducing reliance on continuous human intervention. However, ensuring that such agents learn robust, mission-aligned, and explainable defensive strategies remains a key challenge, particularly in safety-critical military contexts.

In this paper, we investigate graph-based MARL for autonomous cyber defense in a realistic multi-agent enterprise scenario inspired by the TTCP CAGE Challenge 4 [9]. We complement learning-based agents with an expert-informed heuristic baseline that serves both as an interpretable upper bound and as a reference for Explainable Artificial Intelligence (XAI) analysis. We employ XAI techniques, including reward decomposition and action distribution analysis, to systematically assess learned behaviors, diagnose reward misalignment, and relate agent decisions to operational outcomes.

The main contributions of this paper are threefold:

- 1) We propose a graph-based, inductive MARL approach for ACD that achieves performance comparable to state-of-the-art benchmark solutions.
- 2) We design an interpretable heuristic defense agent that establishes a strong expert-informed reference policy for evaluation and analysis.
- 3) We apply XAI techniques to analyze reward structures and agent behavior, revealing structural limitations of the reward/environment design and motivating heuristic-guided learning for future mission-aligned cyber defense systems.

The remainder of this paper is structured as follows. Section II reviews related literature. Section III introduces the network scenario and formal problem formulation. Section IV describes the proposed MARL architecture and XAI methodology. Section V presents and analyzes the experimental results. Finally, Section VI concludes the paper and outlines directions for future research.

II. BACKGROUND

A. Multi-Agent Reinforcement Learning

ACD has emerged as a key research direction for protecting large-scale, mission-critical networks where human-in-the-loop operation is infeasible due to speed, scale, and complexity constraints [2], [10]. In military and coalition environments, networks are typically segmented into multiple security zones

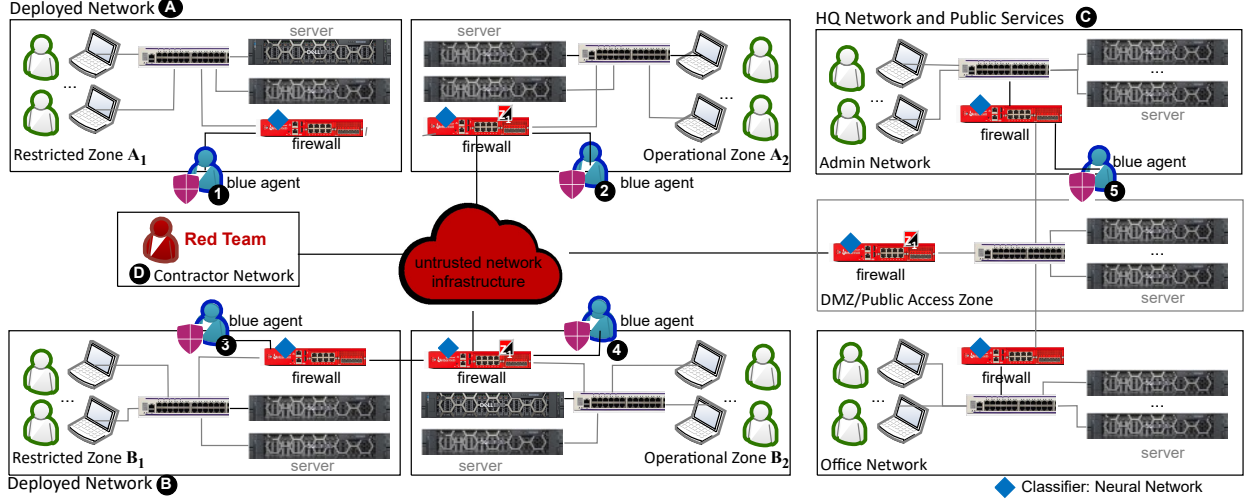


Fig. 1: Scenario with four networks (A-D). Deployed networks A and B each host two blue agents, while HQ network C hosts a fifth. Red agents hosted on contractor network D can use the interconnecting untrusted network infrastructure as entry point.

with heterogeneous operational roles, strict communication policies, and dynamically changing mission priorities [5].

MARL provides a methodology for such settings by distributing decision-making across multiple agents that jointly optimize a shared reward signal. Prior work [2], [10] has demonstrated that decentralizing defensive responsibilities across zones improves scalability, robustness, and fault tolerance compared to monolithic single-agent approaches, particularly in Software-Defined Defense (SDD) contexts where network topology and mission constraints evolve over time.

Recent work has explored hierarchical MARL architectures as a means to address these limitations. Instead of learning a single flat policy over a large and heterogeneous action space, hierarchical approaches decompose the defense task into structured sub-policies that operate at different temporal or functional levels [8]. In this paradigm, lower-level agents execute concrete defensive actions within a zone, while higher-level coordination mechanisms shape priorities and resolve conflicts across zones. Empirical evidence suggests that hierarchical decompositions improve learning stability, accelerate convergence, and enhance cross-scenario transfer by explicitly embedding domain structure into the learning problem. Nevertheless, state-of-the-art MARL approaches are still outperformed by expert-designed heuristic models [9].

B. Explainability in Reinforcement Learning for Cyber Defence

Explainable Reinforcement Learning (XRL) addresses the increasing need for transparency and trust in Deep Reinforcement Learning (DRL) systems, particularly in safety-critical domains such as ACD. Reward decomposition provides a principled approach to explainability by breaking down the total reward into semantically meaningful components. It

enables compact justifications of an agent's action preferences in terms of these components [11].

In this work, we use reward decomposition to systematically examine how different reward components contribute to agent behavior across phases and agents, and to identify structural limitations in the learning setup that are not apparent from aggregate reward signals alone. Explainability techniques have also been explored within the Cyber Autonomy Gym for Experimentation (CAGE) Challenge framework to support the development and analysis of autonomous cyber defense agents.

Prior work [12] applied SHapley Additive exPlanations (SHAP) and reward decomposition to analyze a hybrid DRL agent in a single-agent setting, demonstrating how explainability can reveal conservative defensive strategies and the influence of costly actions. Complementary to this, [10] showed that SHAP can be used to diagnose behavioral weaknesses in multi-agent architectures in CAGE Challenge 4, informing curriculum learning and phase-specific training strategies. Together, these works highlight the practical value of XAI as diagnostic tools during agent development and debugging.

III. NETWORK SCENARIO & PROBLEM

A. Network Scenario

This study utilizes the CAGE Challenge 4 [13] network topology (Fig. 1), comprising four interconnected networks (A, B, C, and D) linked via untrusted infrastructure. Deployed networks A and B each feature two security zones (Restricted and Operational), while Network C integrates three zones (Admin, Office, and DMZ). A total of five blue ACD agents are distributed across these zones: one per zone in A and B, whereas all three zones in C are managed by a single agent. Network D represents an undefended contractor network

serving as the entry point for red attacker agents. Cyber attacks are initiated via three vectors: direct external attacks from network D, phishing mails, or compromised services.

Blue agents must maintain network stability and service availability under strict communication constraints, including limited message sizes and link data rates. This necessitates decentralized decision-making based primarily on local network telemetry. Red agents aim to compromise hosts and disrupt critical services, whereas green agents simulate legitimate traffic on all hosts but introduce stochastic noise via false alarms and security lapses (e.g., phishing susceptibility). The environment operates across three mission phases with evolving connectivity rules and priorities: Phase 1 consists of general low priority operations in all zones, Phase 2a involves increased activity and focus on Deployed Network A, and Phase 2b transitions the focus to Deployed Network B.

B. Problem

To capture the relational structure of the defended network, we represent the environment state as a graph $G = (V, E)$, where V denotes a finite set of nodes representing discrete network entities (e.g., hosts, services, subnets), and $E \subseteq V \times V$ is a set of edges that specify pairwise relations, such as communication paths, containment, or control dependencies.

We consider attributed graphs, in which each node $v \in V$ is associated with a feature vector $f(v) \in \mathbb{R}^d$ that captures security-relevant properties and observations. Collectively, these node features are denoted by $X \in \mathbb{R}^{|V| \times d}$ and are summarized in Table I. Edge attributes, when present, encode additional contextual information such as firewall states or policy constraints.

A graph embedding function

$$\Phi : (V, E, X) \rightarrow Z$$

maps the attributed graph to a latent representation Z , where Z may consist of node-level embeddings, graph-level embeddings, or both. These embeddings jointly encode node attributes and topological structure, providing a compact representation suitable for downstream decision-making.

Embedding functions can be categorized as *transductive* or *inductive*. Transductive embeddings are learned for a fixed set of nodes observed during training and cannot directly generalize to unseen nodes. In contrast, inductive embedding functions learn a parameterized mapping that can generate embeddings for previously unseen nodes or modified graph structures based solely on their features and local connectivity. Inductive representations are therefore essential for cyber defense scenarios in which network size, topology, and host composition vary across deployments and mission phases.

In the considered setting, agents operate on partial and evolving views of the underlying network graph and must generalize across randomized topologies and mission contexts. This motivates the use of inductive graph embeddings that support scalable and adaptive decision-making under partial observability.

Due to strict restrictions on inter-agent communication, partial observability of the global network state, and the dynamic evolution of mission priorities, the considered cyber defense scenario is naturally formulated as a Decentralized Partially Observable Markov Decision Process (Dec-POMDP). Formally, the problem is defined by the 7-tuple $\langle n, \mathcal{S}, \mathcal{A}, \mathcal{P}, \Omega, \mathcal{O}, \mathcal{R} \rangle$ [14], where n denotes the number of defending agents deployed across distinct security zones.

Instead of accessing the global network state $s \in \mathcal{S}$, each blue agent α_i , $i \in \{1, \dots, n\}$, receives a local observation $o_i \in \Omega_i$ derived from its assigned subnet. These observations aggregate partial telemetry about hosts, services, and security-relevant events, including alerts triggered by red or green agents, current firewall configurations, and mission-phase-dependent communication policies. The joint observation $o = \langle o_1, \dots, o_n \rangle \in \Omega$ is generated according to the observation function $\mathcal{O} : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \times \Omega \rightarrow [0, 1]$, which models uncertainty arising from incomplete sensing, delayed effects of durative actions, and stochastic user and attacker behavior.

State transitions are represented by a stochastic transition function $\mathcal{P} : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow [0, 1]$, which captures the effects of defensive actions (e.g., blocking traffic, restoring hosts), adversarial activities (e.g., lateral movement, exploitation), and benign user behavior. Importantly, transition dynamics and operational relevance vary across mission phases, reflecting changing priorities in deployed and headquarters networks.

The reward function $\mathcal{R} : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow \mathbb{R}$ encodes mission impact by penalizing disruptions to service availability, successful adversarial actions, and defensive measures that unnecessarily degrade legitimate operations. The agents jointly aim to maximize the expected discounted return

$$\mathbb{E} \left[\sum_{t=0}^T \gamma^t r_t \right],$$

where r_t denotes the reward at timestep t and $\gamma \in [0, 1]$ is a discount factor emphasizing near-term mission assurance.

IV. METHODOLOGY

A. State-Action Representation

In accordance with the problem formulation in Section III, each blue agent operates under partial observability and receives only local, zone-specific information. Rather than relying on fixed-dimensional feature vectors, we represent the agent's observation directly as an attributed graph

$$\mathcal{O}_i = \langle V_i, E_i, X_i \rangle,$$

where V_i denotes the set of observable entities within the agent's responsibility, E_i captures their relational structure, and X_i contains node- and edge-level attributes derived from local telemetry and mission context. Due to sensing limitations and delayed action effects, the observed graph constitutes only a partial view of the true environment state, i.e., $\mathcal{O}_i \subseteq \mathcal{S}$.

TABLE I: Features tracked in the observation graph.

Node / Edge	Feature	Description
Hosts / Routers	CybORG enums	One-hot vectors for OS type/distro/version/kernel, architecture, and installed patches.
	Role flags	<i>user, server, router, crown_jewel</i> .
	Malicious flags	Recent malicious <i>process / network</i> event indicators attached to the node.
	Aux state	<i>decoy_present</i> , time-since-last <i>Analyse/Restore/Remove</i> , and action cooldown counters.
Open Ports (process/port)	CybORG enums	One-hot for <i>process_name</i> and <i>process_type</i> .
	<i>isDefault</i>	True if present in the initial observation as a default service.
	<i>isEphemeral</i>	True if port > 49152.
	<i>isDecoy</i>	True if created by <i>DeployDecoy</i> .
Files	CybORG enums	One-hot for file vendor/type/version (when observed).
	<i>density</i>	Density value reported by <i>Analyse</i> .
	<i>signed</i>	Signature flag reported by <i>Analyse</i> .
Internet (node)	(none)	Structural node to connect Internet-reachable subnets.
All nodes	Subnet membership	9-dim one-hot indicating the owning subnet/router.
	Node type	One-hot over {Host, Port, File, Internet}.
	Tabular augmentation	Per-host 2-bit flags (“scanned”, “compromised”) injected into host rows from tabular wrapper.
Router-router edges	Firewall & policy	Current allow/deny and the policy target (should-block) bit.

This representation preserves the relational structure of the network and allows the observation space to scale naturally with the size and topology of the defended subnet. Importantly, no assumption is made that agents have full or accurate knowledge of compromise states, attacker paths, or global connectivity beyond what is locally observable or explicitly communicated.

Actions are defined as transformations applied to discrete entities in the graph. Let $V_a \subseteq V_i$ denote the subset of actionable nodes (e.g. hosts) and $E_a \subseteq E_i$ the subset of actionable edges (e.g. inter-subnet connections). Given a finite set of action types $\mathcal{A}$, node-level actions are defined as functions $a_n(v)$ with $v \in V_a$, while edge-level actions are defined analogously as $a_e(u, v)$ with $(u, v) \in E_a$.

This formulation yields a structured action space $\mathcal{A} = \mathcal{A}_V \cup \mathcal{A}_E$, in which actions are parameterized by their target entities rather than by fixed indices. As a result, the effective action space can expand or contract with the number of actionable entities without altering the policy parameterization, which is essential for generalization across networks of varying topology.

The execution of an action induces a corresponding modification of the observed graph, such as changes to node attributes, the addition or removal of edges, or the instantiation of new entities. These graph edits, as illustrated in Tab. II), define the environment transition dynamics from the agent’s perspective and provide a consistent interface between perception, decision-making, and environmental feedback.

B. Graph-Based Proximal Policy Optimization

To learn decentralized defensive policies over the graph-structured observations defined in the previous subsection, we implement a Graph Convolutional Network (GCN) to compute latent representations. Let $\mathcal{O}_i = \langle V_i, E_i, X_i \rangle$ denote the partial observation graph available to agent α_i , where V_i is the set of observed entities, E_i their relations, and

X_i the associated feature matrix. The GCN computes node embeddings iteratively as

$$H^{(\ell+1)} = \sigma \left(D^{-\frac{1}{2}} \tilde{A} D^{-\frac{1}{2}} H^{(\ell)} W^{(\ell)} + b^{(\ell)} \right),$$

where $H^{(0)} = X_i$, $\tilde{A} = A + I$ is the adjacency matrix with self-loops, D is the corresponding degree matrix, $W^{(\ell)}$ and $b^{(\ell)}$ are trainable parameters, and $\sigma(\cdot)$ denotes a nonlinear activation function. Stacking k GCN layers allows each node embedding to combine information from its local k -hop neighborhood, thereby capturing both local security attributes and relational dependencies. The resulting set of embeddings is denoted by $Z = \{z_v \mid v \in V_i\}$. Policy learning is performed using Proximal Policy Optimization (PPO), with both actor and critic sharing the same GCN node embeddings but maintaining separate output heads.

In the inductive formulation, node embeddings must be mapped to action preferences such that the mapping is invariant to the ordering and number of nodes. Let $V_a \subseteq V_i$ denote the subset of actionable nodes. For node-level actions, each embedding z_v with $v \in V_a$ is processed independently by a shared projection function f_π , yielding a vector of unnormalized action scores

$$\ell_v = f_\pi(z_v) \in \mathbb{R}^{|\mathcal{A}_V|},$$

where each component $\ell_{v,j}$ represents the logit associated with applying action a_j to node v . Collecting these vectors across all actionable nodes produces a matrix

$$L \in \mathbb{R}^{|V_a| \times |\mathcal{A}_V|},$$

which is flattened and normalized using a softmax to obtain a probability distribution over all node-level actions. This construction enables full inductivity: adding or removing nodes changes the dimensionality of L without requiring retraining or architectural modification.

TABLE II: Actions and corresponding graph edits.

Prio.	Action	Effect on belief state	Edit level
1	<i>Restore(host)</i>	Remove transient connections to/from the restored host; clear file/process flags linked to compromise; reset host cooldown/state as indicated by the observation.	Node
2	<i>Remove(host)</i>	Delete suspicious session edges associated with the host	Node
3	<i>Analyse(host)</i>	If artefacts are observed, (create or) update a file node and connect it to the analysed host.	Node
4	<i>DeployDecoy(host)</i>	Instantiate a decoy port/process node of the selected service type and attach it to the host; mark port as decoy.	Node
5	<i>Analyse(host)</i>	Proactively analyze lower-priority hosts for undetected malicious events.	Node
N/A	<i>AllowTraffic(from,to)</i>	Set (or create) the router-router edge state to <i>allowed</i> for the specified subnet pair.	Edge
N/A	<i>BlockTraffic(from,to)</i>	Set (or delete) the router-router edge state to <i>blocked</i> for the specified subnet pair.	Edge
N/A	<i>Monitor</i>	Integrate newly observed network events by creating any missing port/process nodes and associations.	Global

The critic network estimates the expected return of the current state by aggregating information across actionable nodes. In a naive inductive formulation, each node embedding is projected to a scalar value and combined using an order-invariant pooling operation. While fully inductive, this approach is limited by the locality of GCN embeddings, which only encode information within a fixed neighborhood.

To incorporate global situational awareness, we augment the model with a learned global graph context vector g . After each GCN layer embeddings of actionable nodes are combined with the current context to update g via an attention-based pooling mechanism:

$$g' = g + \phi_g(\text{POOL}_{v \in V_a} \{\phi_v(g, z_v) \cdot \phi_a(g, z_v)\}),$$

where ϕ_v and ϕ_g are learned projection functions and ϕ_a produces normalized attention weights via a softmax. The pooling operator is order-invariant and utilizes mean- or max-pooling to avoid sensitivity to the number of nodes when evaluating on networks larger than those encountered during training.

The updated global context vector g' is concatenated to each node embedding prior to action evaluation. This ensures that local action decisions are conditioned on both node-specific information and the aggregated defensive state of the network. The actor applies the inductive mapping described above using these augmented embeddings, while the critic estimates the state value directly from g' , reflecting mission-level defensive effectiveness rather than isolated node conditions. We refer to this formulation as an attention-based inductive policy.

The inductive framework naturally extends to edge-level actions. For each actionable edge $(u, v) \in E_a$, an edge representation is constructed as a symmetric function of the corresponding node embeddings, implemented via the Hadamard product $z_u \odot z_v$. A shared projection function

$$\ell_{u,v} = f_e(z_u \odot z_v)$$

maps this representation to logits over edge-level action types. These logits are concatenated with the node-level logits to form a unified action distribution, from which a single action is selected at each timestep.

C. Reward Structure

The reward scheme models a general-sum game in which blue actions affect both service availability for green agents and the impact of red agents. The reward is calculated according to the agent's ability to maintain the operational integrity of the network and neutralize malicious activity. The reward structure reflects the criticality of various hosts and their roles within the mission phases: actions that adversely affect critical mission priorities receive higher penalties than actions affecting non-critical systems.

The dynamic reward function captures how blue and red actions within the network affect the green agents. Green agents can perform three types of actions: (i) *Sleep*, the equivalent of performing no action in that step, (ii) *Local Work*, where the user accesses a service on the host they have a session on, and (iii) *Access Service*, where the user accesses a service on another host. Green agents randomly select one of these actions every turn. Red and blue actions can both cause Local Work and Access Service to fail, resulting in a negative reward. The red agent's *Impact* action can also cause a negative reward, as it signifies an attack on the operational technology (OT) service.

The success or failure of actions, and the subnet in which they occur, are used to calculate a reward score for the given timestep. The cumulative reward across timesteps forms the episode return. Penalties depend on the specific mission phase and the importance of the affected zones: during the pre-mission planning phase (Phase 1), penalties are distributed relatively evenly across zones. During mission-specific phases (Phases 2a and 2b), penalties for failing to maintain services in operational zones increase substantially.

D. Heuristic Benchmark Model

To contextualize the performance of the learned MARL agents and to support the analysis of reward alignment, we implement a heuristic benchmark policy that encodes defensively reasonable behavior derived from expert knowledge. The heuristic agent is not designed to optimize the reward function directly but rather to represent an expert-informed upper bound under the given observation and action constraints. As such, it serves both as a performance reference and as a diagnostic instrument for understanding the incentives induced by the environment and reward design.

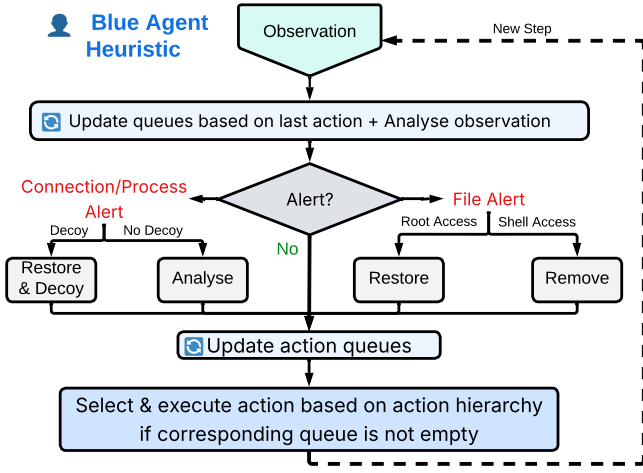


Fig. 2: Heuristic Agent Methodology.

The heuristic policy operates on the same local observations as the MARL agents and applies a combination of reactive and proactive countermeasures in response to detected security events. To maintain temporal consistency beyond a single timestep, the agent tracks unresolved alerts using perception queues that associate observed events with candidate remediation actions. Alerts are categorized according to their semantic type, namely connection-related events, process-related events, and file-related events. Connection and process alerts trigger analysis actions to gather additional host-level evidence, whereas file-related alerts are queued for removal or system restoration depending on the assessed severity and persistence of the artifact.

Action prioritization is implemented by a fixed hierarchy that reflects defensive urgency. Hosts exhibiting correlated indicators, such as simultaneous process or connection alerts together with router-level alerts, are assigned the highest priority for analysis, as these patterns are indicative of active compromise or lateral movement. Alerts involving decoy services are treated as strong indicators of adversarial intent and are therefore escalated to restoration actions to rapidly remove potential attacker footholds. The full decision logic of a single heuristic step is illustrated in Fig. 2, while the corresponding action priority hierarchy is summarized in Table II.

E. Explaining Agent Behavior using XAI

In operational cyber defense scenarios, aggregate performance metrics alone are insufficient to assess whether an ACD agent exhibits defensively meaningful behavior. In particular, identical episode returns may arise from fundamentally different causes, such as unavoidable mission constraints, delayed defensive actions, or systematic reward misalignment. To support a deeper analysis of learned policies and to validate their alignment with expert intuition, we adopt an XAI methodology that focuses on decomposing both rewards and action selection patterns.

The central idea is to make the learning dynamics and resulting behavior of the agents interpretable by explicitly analyzing what they are rewarded for and how they operationalize these incentives through concrete defensive actions. This is achieved through a combination of reward decomposition and action decomposition, applied consistently to both the learned MARL agents and the heuristic reference policy.

Reward decomposition is used to break down the scalar reward signal into semantically meaningful subcomponents that correspond to distinct operational effects, such as service unavailability in specific network zones or the impact of adversarial actions during different mission phases. Rather than treating the total reward as a black box, this decomposition enables a fine-grained attribution of performance outcomes to individual aspects of the environment.

Formally, let the total reward function $R(\mathbf{x})$ for a MARL agent be composed of m weighted components,

$$R(\mathbf{x}) = \sum_{i=1}^m w_i R_i(\mathbf{x}),$$

where $R_i(\mathbf{x})$ denotes the i -th sub-reward associated with a specific operational effect and w_i represents its relative importance. The learning objective is to maximize the expected cumulative reward

$$\mathcal{J}(\pi) = \mathbb{E} \left[\sum_{t=0}^T R(\mathbf{x}_t) \right],$$

where $\mathbf{x}_t$ denotes the state at time t and π denotes the policy executed by the agent. By analyzing the temporal evolution and relative magnitude of each component $R_i(\mathbf{x})$, reward decomposition reveals which aspects of the environment dominate the optimization signal and how these incentives shape the learned behavior across mission phases.

While reward decomposition explains what the agent is incentivized to optimize, it does not directly reveal how the agent attempts to realize these objectives. To address this, we complement the reward-based analysis with action decomposition. During evaluation, the actions selected by each agent are recorded at every timestep and aggregated across episodes and mission phases. Normalizing these results yields empirical action distributions that characterize policy preferences independently of the reward signal. Comparing these distributions between agents allows us to identify systematic behavioral patterns, such as an overemphasis on certain defensive actions or the underutilization of monitoring and analysis behaviors that are critical for early threat detection.

V. RESULTS & DISCUSSION

A. Heuristic Model

Our expert informed heuristic agents were evaluated over 100 episodes with 500 steps per episode, achieving an average return of -132 with a standard deviation of 44.93 . This

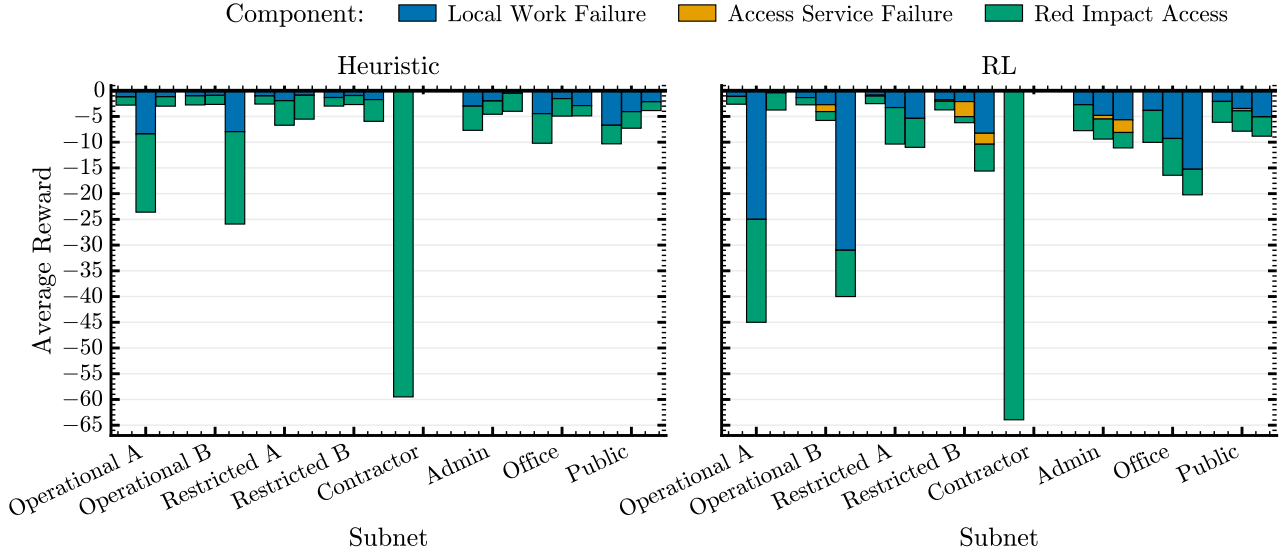


Fig. 3: Average reward by subnet for the Heuristic model (left) and MARL model (right), decomposed into Local Work Failure, Access Service Failure, and Red Impact Access contributions. For each subnet, bars are ordered left-to-right by phase: Phase 1, Phase 2a, then Phase 2b.

performance establishes the heuristic as a strong expert-informed baseline, suitable both as an upper bound for defensively reasonable behavior and as a source of prior knowledge for guided MARL or behavior cloning in early training phases.

Fig. 3 (left) shows the average reward per subnet, decomposed into *Local Work Failure*, *Access Service Failure*, and *Red Impact Access* contributions across all mission phases. The decomposition indicates that the heuristic agents consistently limit high-impact failures, particularly during mission-critical phases, reflecting effective prioritization of operationally relevant assets. Notably, no penalties from *Access Service Failures* are observed, as the heuristic policy avoids unnecessary traffic blocking that would disrupt legitimate mission traffic.

The corresponding action distribution in Fig. 5 highlights a strong preference for monitoring and analysis actions. This behavior reflects a proactive defensive strategy focused on early detection and situational awareness, enabling timely remediation before adversarial activity escalates into mission-impacting events. From an operational perspective, this aligns well with doctrinal cyber defense principles in military networks, where monitoring and controlled responses are essential to maintaining service availability.

B. MARL and XAI Results

To train the graph-based MARL model, five homogeneous blue agents operate concurrently in the network environment. Agents observe the system state through a graph-structured representation and are trained under two configurations in which reward components originating from the contractor network during Phase 0 are either enabled or disabled, in order to assess their influence on learning. In the configuration

reported here, no statistically significant difference between these variants was observed. Training is performed for approximately 10,000 simulation episodes.

The agents are optimized using PPO, where the objective consists of an actor (policy) loss, a critic (value function) loss, and an entropy regularization term. The total loss optimized during training is given by

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{actor}} + \frac{1}{2} \mathcal{L}_{\text{critic}} - \beta \mathcal{L}_{\text{entropy}}, \quad (1)$$

where $\mathcal{L}_{\text{actor}}$ denotes the clipped surrogate policy objective, $\mathcal{L}_{\text{critic}}$ is the mean-squared error of the value function, and $\mathcal{L}_{\text{entropy}}$ encourages exploration with weighting factor β .

During training, the average episode reward is computed by aggregating cumulative rewards across all agents and rollouts within each PPO update, while the mean training loss is recorded to assess optimization stability. Both metrics, shown in Fig. 4, indicate stable convergence without signs of divergence or collapse, demonstrating that the proposed graph-based MARL formulation can be trained reliably under the proposed reward structure.

After training, the MARL agents are evaluated over 100 episodes with 500 steps per episode, achieving an average return of -207.86 with a standard deviation of 46.37 . This performance is closely aligned with the best-performing MARL submissions reported in the official CAGE Challenge 4 analysis [9].

Fig. 3 (right) shows the reward decomposition for the MARL agents across mission phases and network zones. Compared to the heuristic reference, the MARL agents exhibit higher contributions from failure-related reward components, particularly during mission-critical phases 2a and 2b. This

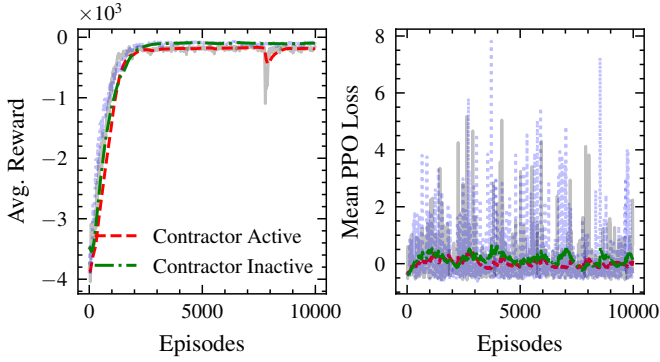


Fig. 4: Training results for MARL models. (Left) Average episode reward over training. (Right) Mean PPO training loss (linear scale). Solid lines indicate exponential moving averages ($\alpha = 0.05$); faint lines show raw values.

indicates increasing difficulty in preserving service availability as operational priorities shift, despite stable training dynamics.

The corresponding action distributions in Fig. 5 provide further insight into this behavior. The MARL agents strongly prioritize connectivity management actions, namely *AllowTrafficZone* and *BlockTrafficZone*, while comparatively underutilizing *Analyse* and monitoring actions. In contrast, the heuristic policy exhibits a more balanced use of information-gathering actions, which likely contributes to its superior performance.

Notably, negative rewards originating from the contractor network, which are largely driven by randomized impact actions of red agents, introduce substantial noise (see Fig. 3) and hinder meaningful comparison of aggregated reward scores. Since blue agents cannot act within or influence the contractor network, these penalties do not reflect defensive performance. To mitigate this effect, we suggest removing the penalties associated with the contractor network during Phase 1 of the evaluation. Under this adjusted setting, the heuristic model attains a mean reward of -67.12 with a standard deviation of 47.53, whereas the MARL model achieves a mean reward of -137.98 with a standard deviation of 44.40.

VI. CONCLUSION & FUTURE WORK

This work investigated a graph-based multi-agent reinforcement learning approach for MARL in a mission-oriented network scenario. By combining inductive graph representations with decentralized Proximal Policy Optimization, we showed that MARL agents can operate on structured, partially observable network states under strict communication constraints and learn non-trivial defensive behaviors.

To validate the learned policies beyond common reward metrics, we applied an explainability methodology based on reward decomposition and action frequency analysis. The results reveal that the MARL agents rely heavily on a limited subset of actions while underutilizing monitoring and analysis behaviors that are essential for early detection and effective defense.

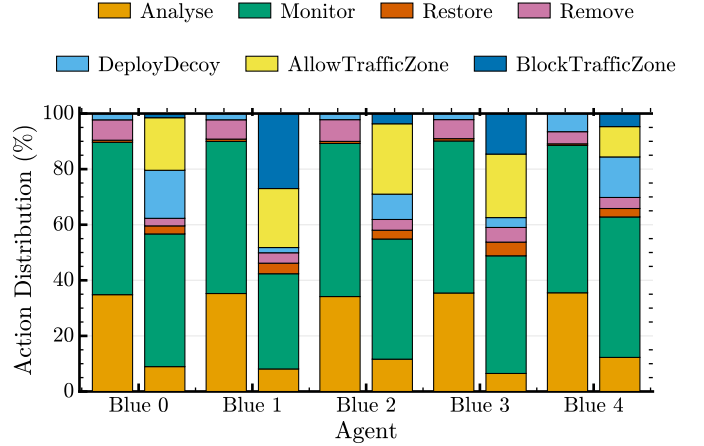


Fig. 5: Action distribution for five Blue agents (0-5). For each agent, the left bar shows the heuristic policy and the right bar shows the RL policy with colors indicating the percentage of actions selected.

In contrast, the heuristic reference policy exhibits a more balanced and proactive strategy, providing an interpretable expert-informed upper bound.

The explainability analysis indicates that the observed performance gap is not primarily caused by insufficient training, but by structural limitations in the current reward and simulation environment design. In particular, reward components associated with the contractor network introduce noise and weakly aligned incentives that can dominate the learning signal and impede the emergence of effective long-term defensive strategies. These findings underscore the importance of reward validation in mission-driven cyber defense environments.

Future work will therefore focus on refining the reward structure, adapting the action space and messaging protocol and on curriculum-based training strategies that progressively expose agents to increasing mission complexity. A particularly promising direction is heuristic-guided reinforcement learning [15], where expert knowledge encoded in the heuristic policy is used to guide exploration and stabilize early learning. Such guidance can be incorporated via auxiliary objective terms while enforcing policy improvement constraints to ensure that heuristic influence is retained only when it leads to measurable performance gains.

ACKNOWLEDGEMENT

This paper was originally presented at the International Conference on Military Communication and Information Systems (ICMCIS), organized by the Information Systems Technology (IST) Scientific and Technical Committee, IST-224-RSY - the ICMCIS, held in Bath, United Kingdom, 12-13 May 2026.

REFERENCES

- [1] J. F. Loevenich, E. Adler, A. Becue, A. Velazquez, K. Wrona, V. Boshnakov, J. Falkcrona, N. Nordbotten, O. L. Worthington, J. Röning, and R. R. F. Lopes, "Training autonomous cyber defence agents: Challenges and opportunities in military networks," in *NATO Science and Technology Organization (STO) Modelling and Simulation Panel*, 2025.
- [2] J. F. Loevenich, E. Adler, R. Mercier, A. Velazquez, and R. R. F. Lopes, "Design of an Autonomous Cyber Defence Agent using Hybrid AI models," in *2024 International Conference on Military Communication and Information Systems (ICMCIS)*, Koblenz, Germany, 2024, pp. 1–10.
- [3] J. F. Loevenich, J. Bode, T. Hürten, L. Liberto, F. Spelter, P. H. L. Rettore, and R. R. F. Lopes, "Adversarial Attacks Against Reinforcement Learning Based Tactical Networks: A Case Study," in *MILCOM 2022 - 2022 IEEE Military Communications Conference (MILCOM)*, 2022, pp. 986–992.
- [4] J. F. Loevenich, T. Hürten, F. Spelter, J. Braun, E. Adler, L. Moxon, and R. R. F. Lopes, "A hybrid model for network condition forecasting to improve qos in tactical manets," in *NOMS 2025-2025 IEEE Network Operations and Management Symposium*, 2025, pp. 1–7.
- [5] J. F. Loevenich and R. R. F. Lopes, "Agentic generative ai for automation of cyber security attack chains in tactical manets," in *2025 IEEE 50th Conference on Local Computer Networks (LCN)*, 2025, pp. 1–7.
- [6] A. Velazquez, R. R. F. Lopes, A. Bécue, J. F. Loevenich, P. H. L. Rettore, and K. Wrona, "Autonomous Cyber Defense Agents for NATO: Threat Analysis, Design, and Experimentation," in *MILCOM 2023 - 2023 IEEE Military Communications Conference (MILCOM)*, 2023, pp. 207–212.
- [7] J. F. Loevenich and R. R. F. Lopes, "On the challenges to adapt to ever-changing network conditions in tactical manets," in *NOMS 2025-2025 IEEE Network Operations and Management Symposium*, 2025, pp. 1–4.
- [8] V. Singh, E. Rathbun, E. Graham, L. Oakley, S. Boboila, and P. Chin, "Hierarchical multi-agent reinforcement learning for cyber network defense," in *International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, 2025.
- [9] M. Kiely, M. Ahiskali, E. Borde, B. Bowman, D. Bowman, D. Van Bruggen, K. Cowan, P. Dasgupta, E. Devendorf, B. Edwards, A. Fitts, S. Fugate, R. Gabrys, W. Gould, H. H. Huang, J. Jacobs, R. Kerr, I. J. King, L. Li, L. Martinez, C. Moir, C. Murphy, O. Naish, C. Owens, M. Purchase, A. Ridley, A. Taylor, S. Farmer, W. J. Valentine, and Y. Zhang, "Exploring the efficacy of multi-agent reinforcement learning for autonomous cyber defence: A CAGE challenge 4 perspective," in *Conference on Artificial Intelligence (AAAI)*, 2025.
- [10] T. Hürten, J. F. Loevenich, F. Spelter, E. Adler, J. Braun, L. Moxon, Y. Gourlet, T. Lefeuvre, and R. R. F. Lopes, "Hierarchical Multi-Agent Reinforcement Learning for Autonomous Cyber Defense in Coalition Networks," in *MILCOM 2024 - 2024 IEEE Military Communications Conference (MILCOM)*, 2024, pp. 176–181.
- [11] Z. Juozapaitis, A. Koul, A. Fern, M. Erwig, and F. Doshi-Velez, "Explainable Reinforcement Learning via Reward Decomposition," in *International Joint Conference on Artificial Intelligence. A Workshop on Explainable Artificial Intelligence.*, 2019.
- [12] S. Kluge, L. Holz, J. F. Loevenich, and R. R. F. Lopes, "Explaining Hybrid RL Agents for Cyber Defense using SHAP and Reward Decomposition," in *MILCOM 2025 - 2025 IEEE Military Communications Conference (MILCOM)*, 2025.
- [13] T. C. W. Group. (2023) *TTCP CAGE Challenge 4* [Online]. Available: <https://github.com/cage-challenge/cage-challenge-4>. [Accessed: 16 January 2026].
- [14] O. Sigaud and O. Buffet, *Markov decision processes in artificial intelligence*. John Wiley & Sons, 2013.
- [15] C.-C. Lee, Z.-W. Hong, and P. Agrawal, "Going beyond heuristics by imposing policy improvement as a constraint," 2025. [Online]. Available: <https://arxiv.org/abs/2507.05328>